

Úvod do problematiky kódování a komprese dat

Zdeněk VAŠÍČEK

Fakulta informačních technologií, Vysoké učení technické v Brně

Brno, Czech Republic

vasicek@fit.vutbr.cz

Organizace kurzu

■ Témata

- základní kompresní techniky (RLE), transformace (BWT, MTF)
- statistické metody kódování a komprese dat (Huffmann, aritmetické kódování)
- slovníkové metody komprese (LZ77, LZ78, GZIP, ...)
- pokročilé metody komprese využívající kontextové modely

■ Ukončení předmětu:

- zkouška hodnocená max. 70 body

■ Podmínka zápočtu:

- vypracování projektu a zisk minimálně 10 bodů

■ Projekt:

- implementace kompresního algoritmu (zadání bude upřesněno),
- složitost implementace odpovídající 26 hodinám,
- hodnocen max. 30 bodů

Obsah

- Základní pojmy (kódování, komprese, redundance)
- Vlastnosti kompresních technik
- Princip kompresních algoritmů
- Měření účinnosti bezztrátových a ztrátových kompresních technik
- Přehled základních kompresních technik
- Vývoj v oblasti bezztrátové komprese, moderní přístupy

Základní pojmy

- **Komprese dat** je proces transformující vstupní (zdrojový) tok dat na jiný datový tok (výstupní, komprimovaný), který má menší velikost.
- Komprese je možné dosáhnout buď
 - snížením redundance v datech (např. užitím vhodnější reprezentace), nebo
 - odstraněním redundance v datech.
- **Redundance** odpovídá relativnímu množství dat přenesených zbytečně na jeden symbol, které nemá vliv na interpretaci výsledku (intuitivní definice, formální viz dále)
- Příklady redundance:
 - *Redundance kódování*: Předpokládejme, že máme systém generující tok tvořený symboly {A,B,C,D}, který je kódován pomocí 8-bitového ASCII kódu. Redundance je minimálně 75%, neboť použitím 2-bitové reprezentace lze ušetřit 6 bitů na každých 8 bitů
 - *Gramatická redundance jazyka*: “velký strom je zelený” vs. “velké stromy jsou zelené”. Všechna slova jsou ohnutá při vyjadřování mn. č. Stejně množství informace může poskytnout také “velký stromy být zelený”

Redundance

■ Další příklady redundance:

- *Abecední redundance jazyka*: V anglickém textu se písmeno E vyskytuje nejčastěji. Je proto je výhodné pro reprezentaci anglických textů použít kód s proměnnou délkou kódového slova a přiřadit písmenu E nejkratší kód.
- *Kontextová redundance*: Existence vzájemné závislosti výskytu symbolů, např. za písmenem Q následuje v angličtině nejčastěji U. V obrazových a zvukových záznamech mají sousední vzorky podobnou hodnotu.
- *Perceptivní redundance*: konzumentem obrazových a zvukových dat je typicky člověk, jehož smyslové orgány mají omezené schopnosti. Např. člověk neslyší signály s vyššími kmitočty jako např. pes. Takové signály proto není nutné v původním datovém toku zachovávat.

■ Náhodná data postrádají redundanci a komprese datových toků tvořených náhodnými daty je z principu nemožná. Viz komprese šifrovaných dat ...

- **Základním principem komprese** snížením redundance je přiřadit často se vyskytujícím symbolům (frázím) *krátké kódy* a méně se vyskytujícím *kódy dlouhé*.
- Redundanci je zapotřebí vyjádřit objektivně – viz dále teorie informace.

Základní pojmy

- ***Kódování dat*** je proces vzájemně jednoznačného přiřazení symbolů jedné množiny symbolům druhé množiny. Tato definice implicitně předpokládá *blokový kód (viz dále)*.
- Termín *kód* se používá ve dvou významech
 - předpis pro vzájemně jednoznačné přiřazení mezi kódovanými symboly a kódovými kombinacemi (např. binární kód, přímý kód, Huffmanův kód), nebo
 - jako označení vlastní kódové kombinace.
- **Techniky kódování z pohledu redundance:**
 - *nemění* redundanci kódu (vzácně se vyskytující, např. NRZ)
 - *zvyšují* redundanci kódu (typické pro kódy pro opravu chyb, např. Hammingův kód)
 - *snižují* redundanci (typické pro kompresi dat, např. Morseův kód)
- **Kódy zvyšující redundanci typicky zlepšují nějakou jinou vlastnost**
 - odolnost proti chybám (sudá/lichá parita, Hammingův kód)
 - elektrické vlastnosti při přenosu (Manchester, ...)

Kompresní techniky

- Kompresním algoritmem (technikou) se typicky rozumí dva algoritmy
 - *kompresní algoritmus (kompresor)*, který ze vstupního toku X generuje výstupní tok Y reprezentovaný typicky na méně bitech
 - *dekompresní algoritmus (dekompresor)*, který provádí rekonstrukci toku Y buď na původní tok X nebo na jeho aproximaci $\tilde{X}$



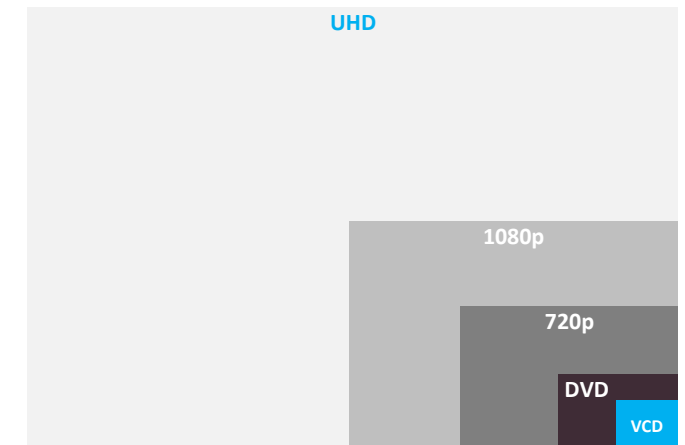
- Podle vztahu rekonstruovaného a vstupního toku rozlišujeme mezi
 - *bezeztrátovým algoritmem*, který z toku Y obnoví původní tok X (typicky případ komprese textu)
 - *ztrátovým algoritmem*, který z toku Y získá aproximaci původního toku $\tilde{X}$ (typicky případ komprese obrazových a zvukových dat, kde lze tolerovat jistou nepřesnost).
- V případě ztrátového algoritmu je nevratně odstraněna část informace, která má v daném kontextu malý význam (např. vysoké frekvence, jasové detaily nepostřehnutelné lidským okem).

Význam a role komprese dat

- Techniky komprese dat jsou motivovány zejména potřebou zefektivnit proces zpracování informace.
- Typicky se jedná o následující problémy:
 - efektivní uchovávání dat,
 - efektivní využití propustnosti přenosové linky,
 - redukce doby přenosu.
- U ztrátových algoritmů řešíme v praxi typicky jeden ze dvou problémů
 - Máme dáno omezení v podobě maximální přenosové rychlosti / kapacity úložiště a cílem je zkomprimovat data tak, abychom vyhověli tomuto omezení a současně maximalizovali kvalitu (tzv. *distortion-rate problem*)
Př.: komprese telefonního hovoru, video konference
 - Máme dán požadavek na kvalitu a cílem je dosáhnout co nejmenšího počtu bitů za sekundu (tzv. *rate-distortion problem*).
Př.: komprese audio signálu pro dosažení CD-kvality, MP3 VBR

Význam a role komprese dat

- Cena úložiště v poslední době dramaticky klesla, současně stoupla kapacita přenosových linek, avšak nároky na úložiště i propustnost přenosových linek se mnohonásobně zvýšily (viz např. online streaming na mobilních zařízeních).
- Problematika komprese dat je proto stále aktuální. Snahou je využít dostupných prostředků na maximum při zachování minimální ceny za MB.
- Bez existence účinných kompresních technik by byla řada technologických vylepšení těžce realizovatelná (viz např. zvýšení rozlišení televizního vysílání nebo digitálního záznamu)
- Komprese použitá na video nosičích:
 - Video CD (1993) 352x288 bodů, MPEG1
 - DVD (1996) 720×576 bodů, MPEG2
 - Blue-ray (2006) 3840×2160 bodů (SD/HD/4K/UHD), MPEG4/HEVC



Vlastnosti kompresních technik

- Jednotlivé kompresní techniky mají různé vlastnosti závislé na způsobu redukce redundance, implementace kompresního a dekompresního algoritmu, způsobu zpracování vstupního toku, apod.
- Časová / paměťová složitost kompresní metody
 - *Symetrické kompresní metody* mají stejnou náročnost komprimace a dekomprimace, stejnou časovou složitost. Z toho vyplývá, že používají ke komprimaci a dekomprimaci stejného algoritmu.
 - *Asymetrické kompresní metody* mají rozdílnou náročnost komprimace a dekomprimace, rozdílnou časovou složitost (někdy i paměťovou složitost). Z toho vyplývá, že používají ke komprimaci a dekomprimaci jiného algoritmu.
- Způsob zpracování vstupního toku
 - *Jednoprůchodové kompresní metody* provádějí kompresi dat v jednom jediném průchodu. Mezi jednoprůchodové kompresní metody patří všechny adaptivní kompresní metody (např. LZ78, PPMC, ...)
 - *Dvouprůchodové kompresní metody* potřebují ke kompresi dvou průchodů komprimovaného toku. První průchod se většinou používá k výpočtu četností výskytu jednotlivých symbolů. Při druhém průchodu se ke komprimaci využívá statistik vypočítaných v průchodu prvním. Mezi metody dvouprůchodové patří metody semi-adaptivní.

Vlastnosti kompresních technik

■ Režim zpracování vstupního toku u jednorůchodových metod

- Většina kompresních metod pracuje v *proudovém režimu*, kdy kodér bere postupně symbol nebo několik symbolů, tyto zpracuje a tak postupuje, až narazí na konec vstupního toku.
- Některé metody však pracují v *blokovém režimu*, kdy se vstupní tok čte po blocích a každý blok se kóduje odděleně. Velikost bloku pak musí být nastavitelný parametr, protože se tím může výrazně ovlivnit účinnost metody.

■ Princip komprese (redukce redundance)

- *Statistické kompresní metody* využívají ke komprimaci četnosti výskytů jednotlivých symbolů ve zdrojovém textu. Většinou jde o dvouprůchodové kompresní metody. Př. Huffmanovo kódování, Aritmetické kódování
- *Perceptivní komprese*. Ztrátový kompresor může využívat specifických vlastností dat. Může pak odstraňovat data, jejichž nepřítomnost člověk nepozná. Zde se využívá znalostí o lidském vnímání zvuku a obrazu, tedy o *psychoakustické* a *psychovizuální percepci*. Př. JPEG, MP3
- *Kontextové kompresní metody* používají při kompresi informací o kontextu komprimovaného symbolu, tj. tvar zkomprimované zprávy záleží také na okolních symbolech původní zprávy. Př. PPMC

Vlastnosti kompresních technik

■ Fyzické a logické kompresní metody

- Většina kompresních metod je *fyzických*. To znamená, že pracují se vstupním bitovým tokem a nevyužívají jeho členění na byty, slova, pixely atd. Prostě nahrazují vstupní datový tok výstupním.
- Některé kompresní metody jsou *logické*, protože pracují s datovými položkami vstupního toku a nahrazují je kratšími položkami. Takové metody jsou v zásadě specializované a mohou zpracovávat pouze speciální typ dat.

■ Techniky je možné dále dělit na datově specifické a datově univerzální

- *datově specifická komprese* je navržena s cílem dosáhnout co nejvyšší účinnosti při použití na datech, pro která byla optimalizována

Příklad: bezztrátová metoda FLAC umožňuje redukovat audio data uložená na CD v průměru o 40 až 50% tím, že zohledňuje vlastnosti charakteristické pro audio data. Při použití obecné techniky jako je DEFLATE (viz zip/gzip) se dosahuje pouze 10-20% redukce.

Princip komprese

- Kompresní algoritmy jsou typicky navrhovány jako dvoufázové.
- První fáze komprese: *tvorba modelu (angl. modeling)*
 - Proces založený na analýze charakteristik vstupního toku jehož cílem je nalezení nějakého vzoru ve vstupním datovém toku.
 - Nalezený vzor (tj. vyšší pravděpodobnosti určitých symbolů/frází) následně umožní kompaktnější reprezentaci vstupního toku.
- Druhá fáze komprese: *kódování (angl. coding)* vycházející ze znalosti modelu
 - Využívá existence vzoru ke generování kódu.
 - Základním principem je zkrátit často se vyskytující fráze a prodloužit méně se vyskytující fráze.
- Rozdíl mezi symboly generovanými modelem a symboly vstupního toku se nazývá reziduum (angl. *residual*).
- Je zřejmé, že návrh kompresního algoritmu má mnoho stupňů volnosti. Neexistuje jeden přístup, který by vyhovoval všem komprimovaným datům.

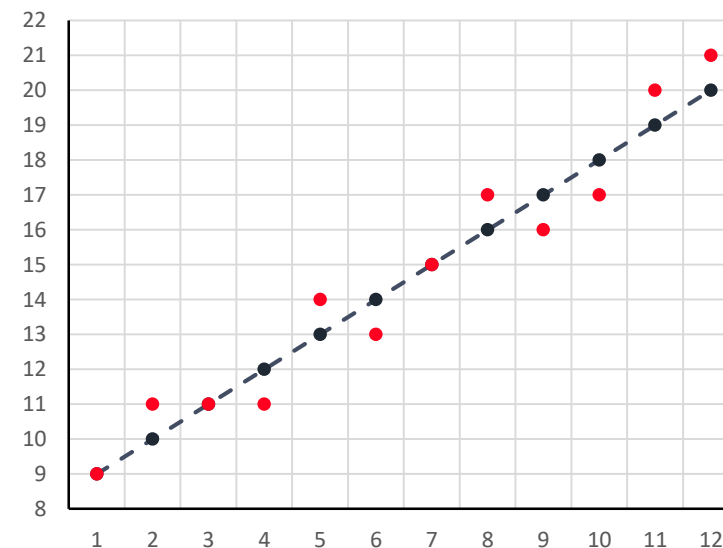
Příklady modelů a kódování – příklad 1

- Uvažujme následující sekvenci hodnot $X = \{x_1, x_2, x_3, \dots\}$:

9	11	11	11	14	13	15	17	16	17	20	21
---	----	----	----	----	----	----	----	----	----	----	----

- Binární reprezentace: 5 bitů na vzorek, tj. 60 bitů celkem. Pokud využijeme znalosti struktury dat, můžeme data zakódovat pomocí méně symbolů.
- Pokud vykreslíme data do grafu, zjistíme, že mají rostoucí tendenci.
- Modelem může být přímka daná rovnicí $\hat{x}_n = n + 8$ pro $n = 1, 2, \dots$
- Rozdíl mezi daty a modelem (tj. reziduum) je sekvence $e_n = x_n - \hat{x}_n$, která pro náš příklad odpovídá následující posloupnosti hodnot:

0 1 0 -1 1 -1 0 1 -1 -1 1 1



Příklady modelů a kódování – ad příklad 1

- Reziduum je tvořeno pouze třemi symboly (hodnotami) a sice -1 0 1 .
- K zakódování tří hodnot můžeme použít běžný 2-bitový binární kód
 - např. 00 přiřadíme -1 , 01 přiřadíme 0 , 10 přiřadíme 1
- Vstupní sekvenci tedy můžeme zkomprimovat na 24 bitů. V praxi však bude nutné navíc uložit i použitý model.
- Shrnutí:
 - Redundance (typ struktury dat), která je přítomna v těchto datech vede na jednoduchý princip. Jakmile jsme jej odhalili, můžeme jej použít k predikci hodnot vstupní sekvence a zakódovat pouze reziduum (rozdíl mezi predikovanou a skutečnou hodnotou).
 - Rozdíl mezi hodnotou predikovanou modelem a skutečnou hodnotou můžeme zakódovat přesně (bezeztrátová komprese) nebo s jistou mírou nepřesnosti (ztrátová komprese).

Příklady modelů a kódování – příklad 2

- Uvažujme následující sekvenci hodnot $X = \{x_1, x_2, x_3, \dots\}$:

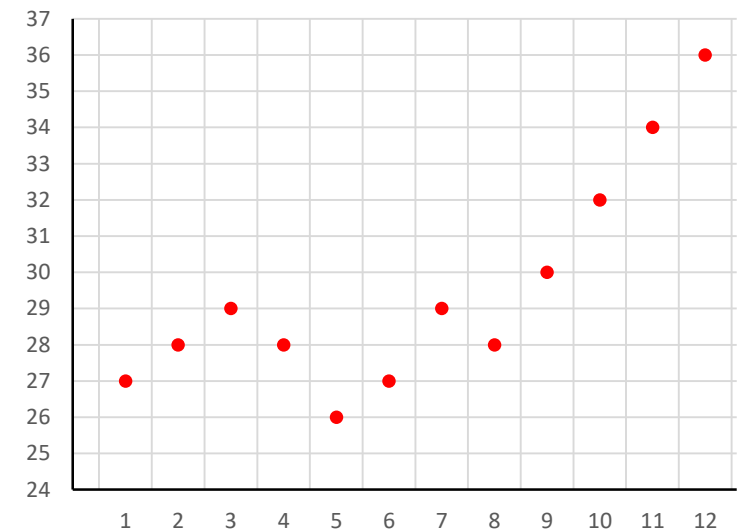
27	28	29	28	26	27	29	28	30	32	34	36
----	----	----	----	----	----	----	----	----	----	----	----

- V porovnání s předchozím případem není patrné, že by data sledovala nějaký jednoduchý princip.
- Sousední hodnoty jsou však velice podobné. Můžeme tedy transformovat sekvenci X na prvotní hodnotu následovanou rozdílem mezi sousedními vzorky, tj.

$$y_n = \begin{cases} x_n - x_{n-1} & \text{pro } n > 1 \\ x_1 & \text{pro } n = 1 \end{cases}$$

- Tím dostáváme následující sekvenci:

27	1	1	-1	-2	1	2	-1	2	2	2	2
----	---	---	----	----	---	---	----	---	---	---	---



Příklady modelů a kódování – ad příklad 2

- Podobně jako v předchozím případě se nám podařilo zredukovat počet různých hodnot, které je nutné zakódovat. Pokud kompresor i dekompresor znají použitý model, pak stačí poslat pouze hodnotu prvního vzorku sekvence následovanou rozdíly.
- Dekompresor pracuje v reverzním režimu. Během rekonstrukce se dekóduje rozdíl uložený v komprimované sekvenci a ten se následně přičte k předchozí hodnotě.
- Komprese bude dosažena za předpokladu, že rozdíl je možné reprezentovat menším počtem bitů.
- Shrnutí:
 - Pokud jsou hodnoty komprimované sekvence podobné, pak je možné dosáhnout komprese tím, že budeme kódovat rozdíl vůči předchozí hodnotě.
 - Technika, která využívá znalosti předchozích hodnot sekvence, provádí predikci aktuální hodnoty a kóduje rozdíl mezi skutečností se nazývá *predikční kódování*.

Příklady modelů a kódování – příklad 3

- Zcela odlišným typem redundance je *statistická redundance*. V praxi se často vyskytují zdroje dat, které generují určité symboly častěji nežli jiné. V tomto případě lze s výhodou využít binárních kódů s proměnnou délkou kódu.
- Předpokládejme, že máme k dispozici následující sekvenci 13 symbolů:

a	b	r	a	c	a	d	a	b	r	a	c	a
---	---	---	---	---	---	---	---	---	---	---	---	---

- V případě použití běžného binárního kódu bychom potřebovali 3 bity na symbol, tj. celkem 39 bitů.
- Pokud však použijeme kód s proměnnou délkou značek (viz tabulka), můžeme sekvenci zakódovat na 27 bitech, tj. $27/13 = 2.08$ bitů na symbol.

Symbol	Výskytů	Kód
a	6	0
b	2	101
c	2	110
d	1	100
r	2	111

Příklady modelů a kódování – příklad 4

- Kromě statistické redundance se často vyskytuje i redundance na úrovni frází (slov), která je typická pro text. Této redundanci lze využít tak, že se zkonstruuje seznam frází, který tvoří model, a při kompresi kódujeme pozici fráze v seznamu (modelu). Tento princip komprese se obecně označuje jako *slovníková kompresní metoda*.

- Předpokládejme stejnou vstupní sekvenci jako v předchozí ukázce

a	b	r	a	c	a	d	a	b	r	a	c	a
---	---	---	---	---	---	---	---	---	---	---	---	---

- Pokud kodér a dekodér sdílí stejný slovník 512 položek (viz tabulka), můžeme vstup zakódovat pomocí tří indexů.

abraca	+	d	+	abraca
410		4		410

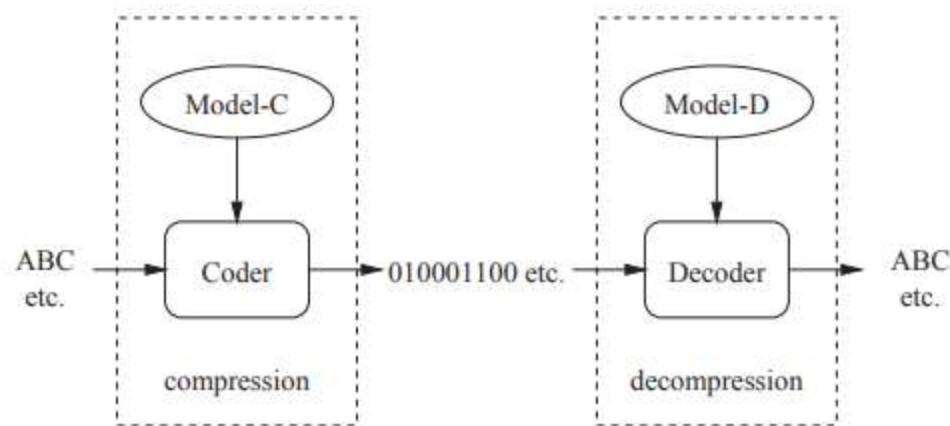
- Při použití běžného binárního kódu (9 bitů na index) lze sekvenci zakódovat pomocí 27 bitů.

Fráze	Index
a	1
...	...
d	4
...	...
abraca	410
...	511

Klasifikace kompresních metod podle typu modelu

■ *Statické kompresní metody*

- Používají při kompresi statický model dat, který se nemění v závislosti na komprimovaných datech.
- Typicky se jedná o četnosti výskytu jednotlivých symbolů / frází, které jsou na základě obecných pravidel vypočítány a používány pro kompresi všech dat.
- Kodér i dekodér pracují se stejným modelem dat. Model se nepřenáší se zakódovanými daty a musí být znám před začátkem komprese.

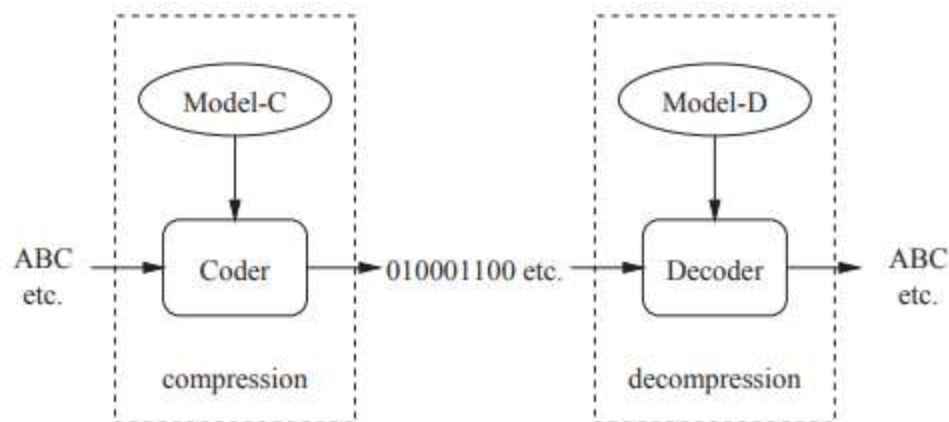


- Př.: Morseova abeceda, slovníková metoda se statickým slovníkem

Klasifikace kompresních metod podle typu modelu

■ *Semi-adaptivní kompresní metody*

- Vytvářejí model dat v závislosti na komprimovaných datech.
- Model je nutné přenést spolu se zkomprimovanou zprávou.
- Na straně dekodéru se nejprve načte uložený model dat, která se následně použije pro rekonstrukci dat.
- Semi-adaptivní kompresní metody patří většinou mezi dvouprůchodové kompresní metody - první průchod textu je použit k vytvoření modelu dat.

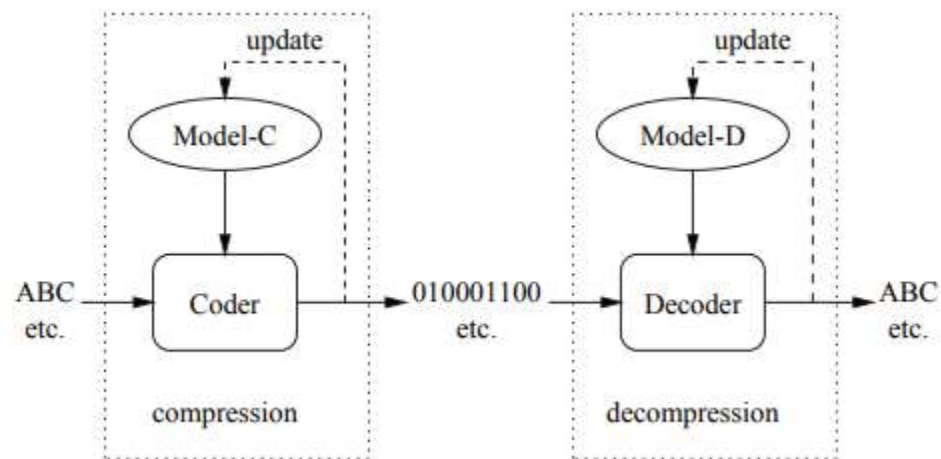


- Příklad: Huffmanovo kódování, Aritmetické kódování, slovníková metoda s dynamickým slovníkem

Klasifikace kompresních metod podle typu modelu

■ *Adaptivní kompresní metody*

- Vytvářejí model dat potřebný při kompresi v závislosti na komprimovaných datech.
- Tento model není nutné přenášet společně se zkomprimovanými daty, neboť při dekompresi si dekodér vytváří stejný model dat jako kodér při kompresi.
- Kodér při komprimaci vždy nejprve zakóduje další symbol / frázi a teprve následně upraví svůj model podle načteného symbolu. Tento postup umožní dekodéru vytvářet si svůj identický model. Kdyby kodér nejprve upravil model a následně zakódoval vstupní jednotku, dekodér by přečetl zakódovanou jednotku podle modelu, který nezná.
- Většina používaných kompresních metod je adaptivní.
- Příklady: Adaptivní aritmetické / Huffmanovo kódování, PPMC, slovníková metoda LZW



Klasifikace kompresních metod

- *Řád modelu* odpovídá počtu symbolů předcházejících právě zpracovávaný symbol, které jsou uvažovány společně s právě zpracovávaným symbolem
 - model řádu 0 nebere ohled na předchozí symboly, pro výpočet statistik je rozhodující pouze aktuálně zpracovávaný symbol (např. Huffmanovo kódování)
 - *modely vyššího řádu* berou v potaz i předchozí symboly (např. PPM)
- Kompresní metody mohou být děleny také dle vlastností použitého kódování:
 - *Fixed-to-fixed*: každý symbol před i po kompresi má pevný počet bitů
Př. A:00, B:01, C:10, D:11 (binární kód)
 - *Fixed-to-variable*: symbol před kompresí má pevný a po kompresi variabilní počet bitů
Př. A:0; B:10; C:101; D:0101 (Huffmanův kód)
 - *Variable-to-fixed*: sekvence symbolů je reprezentována pevným počtem bitů
Př. ABCD:00; ABCDE:01; BC:11 (Např. slovníková metoda s pevným slovníkem)
 - *Variable-to-variable*: různě dlouhé sekvence symbolů jsou reprezentovány variabilním počtem bitů.
Př. ABCD:0; ABCDE:01; BC:1; BBB:0001.

Měření účinnosti a výkonnosti kompresních algoritmů

- Kompresní algoritmy mohou být hodnoceny různými způsoby. Můžeme měřit například:
 - relativní složitost algoritmu,
 - paměťové nároky algoritmu,
 - rychlost komprese a dekomprese,
 - míru dosažené komprese,
 - odlišnost vstupního a rekonstruovaného datového toku (ztrátové metody).
- Pro určení míry dosažené komprese (kompresní výkonnosti) se typicky používá několik veličin:
 - *kompresní poměr* (compression ratio),
 - *kompresní faktor* (compression factor)
 - *kompresní zisk* (compression gain)
- Kompresní faktor, případně kompresní poměr, jsou jedny z nejpoužívanějších metrik pro hodnocení výkonnosti komprese. Pozor, není používáno v literatuře jednotně.
- *Rychlost komprese* lze měřit v *počtu cyklů na byte* (cpb). Je to střední počet strojových cyklů potřebných ke kompresi jednoho bytu.



Měření účinnosti a výkonnosti kompresních algoritmů

- **Kompresní poměr** je definován jako poměr velikosti výstupního (tj. komprimovaného) toku ku velikosti vstupního toku

$$\text{kompresní poměr} = \frac{\text{velikost výstupního toku}}{\text{velikost vstupního toku}}$$

- Kompresní poměr udává, kolik procent původního objemu zaujímají komprimovaná data.
- Čím menší je kompresní poměr, tím lépe je schopen algoritmus redukovat redundanci.
- V praxi může nastat i situace, kdy kompresní poměr je větší než 100% - tzv. *expanze*, kdy nebyla odhalena redundance v datech, ale bylo nutné přidat určitá metadata tak, aby bylo proveditelné dekódování.
- Příklad: Předpokládejme, že máme obraz 256x256 8-bitových pixelů. Po aplikaci kompresního algoritmu má výstupní tok 16 384 bytů.
 - kompresní poměr je 16384:65536 tj. 1:4 což odpovídá 0.25
 - můžeme též říci, že dosažený kompresní poměr je 25%

Měření účinnosti a výkonnosti kompresních algoritmů

- *Procentuální úspora* je definována jako

$$\text{procentuální úspora} = \frac{\text{velikost vstup.toku} - \text{velikost výstup.toku}}{\text{velikost vstupního toku}} \%$$

- Procentuální úspora říká, kolik procent původní velikosti uspoříme kompresí
- Tutéž hodnotu získáme i jako
$$\text{procentuální úspora} = 1 - \text{kompresní poměr}$$
- Příklad: Předpokládejme, že máme obraz 256x256 8-bitových pixelů. Po aplikaci kompresního algoritmu má výstupní tok 16 384 bytů.
 - procentuální úspora je $(65536 - 16384):65536 = 0.75$, tj. 75%
 - uspořili jsme 75% původního objemu dat
- Někdy se též udává výkonnost jako **průměrný počet bitů na znak** (bpc), typické pro bezztrátové metody pro kompresi textu.

Měření účinnosti a výkonnosti kompresních algoritmů

- Převrácená hodnota kompresního poměru se označuje jako *kompresní faktor*

$$\text{kompresní faktor} = \frac{\text{velikost vstupního toku}}{\text{velikost výstupního toku}}$$

- Tato míra je pro mnohé srozumitelnější nežli kompresní poměr, protože čím je tato míra větší, tím lepší je komprese protože kompresní faktor udává, kolikanásobně se podařilo zredukovat původní objem dat.
- V případě kompresního faktoru hodnoty větší než 1 znamenají dosažení komprese a hodnoty menší než 1 expanzi.
- Příklad: Předpokládejme, že máme obraz 256x256 8-bitových pixelů. Po aplikaci kompresního algoritmu má výstupní tok 16 384 bytů.
 - kompresní faktor je 65536:16384 tj. 4:1
 - množství dat potřebných na uchování informace se podařilo čtyřikrát zmenšit

Měření účinnosti a výkonnosti kompresních algoritmů

- *Kompresní zisk* je definován výrazem

$$\textit{kompresní zisk} = 100 \log_e \frac{\textit{referencni velikost}}{\textit{komprimovana velikost}}$$

a používá se k vyhodnocení dvou a více kompresních algoritmů.

- Zde je referenční velikost buď velikost vstupního toku, nebo velikost komprimovaného toku vytvořeného nějakou standardní bezeztrátovou kompresní metodou. Protože platí, že $\log_e 1 + x \approx x$ pro malé hodnoty x , malá změna kompresního zisku odpovídá změně kompresního poměru.
- Jednotka kompresního zisku se nazývá *procentuální logaritmický poměr* (percent log ratio) a značí se o/o.

Měření účinnosti a výkonnosti kompresních algoritmů

- Hodnocení výkonnosti se provádí typicky nad korpusem sestaveným speciálně k testování kompresních programů
- **korpus *Calgary* (1987)**
 - 18 standardně používaných souborů (textové, obrazové a binární soubory, cca 3,2 MB)
- **korpus *Canterbury* (1997)**
 - alternativa ke korpusu Calgary, 11 souborů 2.5MB.
- **korpus *Silesia* (2003)**
 - databázové soubory, binární soubory exe, archívy, texty (díla Dickense, Reymont, slovník Webster), soubory HTML, medicínské obrázky, které se nesmí komprimovat ztrátově
 - soubory od 6 to 51MB, celkem 212 MB dat
- **Korpus LTCB (Large Text Compression Benchmark) (2011)**
 - 1GB (enwik9) a 100MB (enwik8) textu z Wikipedie

Měření výkonnosti u ztrátových algoritmů

- V případě ztrátové komprese se rekonstruovaný tok $\tilde{X}$ liší od původního datového toku X . Je proto nutné jistým způsobem kvantifikovat rozdíl, který se běžně nazývá zkreslení (angl. distortion). Platí, že $\tilde{X} = X + E$, kde E je chyba rekonstrukce (residuum) po kompresi.
- Ztrátové techniky jsou používány pro kompresi původně analogových signálů jako je zvuk či video, kdy příjemcem těchto dat je člověk a je proto obtížné vyjádřit zkreslení matematicky. Často se využívají metriky typu PSNR, MSE, apod.
- Typicky se posuzuje *věrnost* (angl. fidelity) a *kvalita* (angl. quality), v obou případech se zkoumá, jak blízko jsou si $\tilde{X}$ a X .
 - *věrnost* zkoumá, jestli jsme schopni rozlišit $\tilde{X}$ a X . Pokud ne, můžeme prohlásit algoritmus za vizuálně bezztrátový. K hodnocení je možné použít modely lidského vnímání, které jsou založeny na schopnosti rozlišit změnu (viz např. omezený počet odstínů šedé).
 - *kvalita* je subjektivní a týká se preference jednoho obrazu oproti jinému. Můžeme mít např. dva obrazy, které jsou různé (originální a zkreslený vlivem šumu) a přitom preferovat zkreslenou variantu.

Proč má smysl se zabývat kompresí

- I přes více než 40 let historie kompresních technik (viz. příchod LZ77, LZ78 apod.) je problematika komprese dat stále aktuální.
- Výhodnost metod se mění s dostupností výpočetních prostředků a dříve nezajímavé techniky jako jsou např. výpočetně náročné modely vyššího řádu se dostávají do popředí (viz analogie s využitím neuronových sítí).
- Z teoretického hlediska není problematika návrhu uzavřena
 - Návrh kompresního algoritmu je multikriteriální problém, kdy je nutné brát ohled nejen na kompresní poměr, ale také na další parametry jako jsou paměťové nároky, výpočetní složitost apod. Často je také zapotřebí zohlednit i další parametry týkající se HW implementace jako je složitost kodéru/dekodéru, příkon, apod.
 - Ztrátová komprese přidává další stupeň volnosti návrhu – kvalitu / věrnost.
 - Proces bezztrátové komprese a dekomprese je možné stále zdokonalovat. Viz například soutěž Hutter Prize (prize.hutter1.net) - 50 000 € Prize for Compressing Human Knowledge – “Compress the 100MB file enwik8 to less than the current record of about 16MB”
 - Z teoretického hlediska neexistuje ideální bezztrátový kompresní algoritmus – viz dále.

- Jaký je teoretický nejvyšší kompresní poměr, který lze dosáhnout pomocí bezztrátového kompresního algoritmu?
- Tvrzení 1: *Neexistuje algoritmus, který by byl schopen zkomprimovat všechny vstupní soubory byť by se mělo jednat o redukci o jediný bit.*
- Neformální důkaz:
 - Předpokládejme, že existuje algoritmus C, který by byl schopen vždy zredukovat vstup. Pokud by neplatilo tvrzení 1, mohli bychom aplikovat algoritmus rekurzivně na vstup X, tj. $C(C(C(C(C(C.....C(X).....))))))$.
 - Po nekonečně mnoho krocích bychom ale tím pádem museli dostat sekvenci nulové délky. Pak by ale nebylo možné jednoznačně rekonstruovat původní data.
- Pokud některý tok bude zkomprimován, jiný musí být prodloužen.

Limity bezetrátové komprese

- Jaký je teoretický nejvyšší kompresní poměr, který lze dosáhnout pomocí bezetrátového kompresního algoritmu?
- Tvzení 2: *Neexistuje žádný algoritmus, který by byl schopen dosáhnout komprese pro více než 50% všech vstupů byť by mělo jít o redukci o jediný bit.*
- Neformální důkaz:
 - Bezeztrátovou kompresi lze chápat jako jednoznačné mapování mezi zkomprimovaným a původním vstupním tokem. V opačném případě by nebylo možné jednoznačně rekonstruovat zkomprimovaný tok.
 - Komprese vstupu o délce n bitů o alespoň jeden bit znamená existenci mapování sekvencí délky n bitů na sekvence délky maximálně $n-1$ bitů. Takových sekvencí existuje 2^n . Sekvencí o jeden bit kratších existuje právě 2^{n-1} , tzn. zkomprimovat by bylo možné kvůli jednoznačnosti dekódování právě $(2^n - 2^{n-1}) / 2^n = 1/2$ všech vstupů.
- V případě komprese na úrovni bytů není možné zkomprimovat více než $1/256$ (tj. 0.3%) všech možných vstupních toků.

Poslední vývoj v oblasti kódování

Finite State Entropy

- Moderní komprese dat je založena na dvou přístupech entropického kódování Aritmetickém nebo Huffmanovu kódování.
 - Huffmanovo kódování je mnohem rychlejší avšak aproximuje pravděpodobnosti výskytu symbolů pomocí mocnin 2, což v určitých případech vede na špatný kompresní poměr.
 - Aritmetické kódování umožňuje zachytit pravděpodobnosti téměř přesně, je možné dosáhnout teoretického limitu (Shannonova entropie), avšak za cenu vyšších výpočetních nároků.
- V roce 2014 se v oblasti komprese a kódování objevila nová metoda Finite State Entropy (FSE) založená na Asymmetric numeral systems (Jarosław Duda, 2007-2013), která je rychlostí a složitostí implementace srovnatelná s Huffmanovým kódování a přitom dosahuje podobného kompresního poměru jako Aritmetické kódování.
- Použití: Apple LZFSE – 2015, Facebook ZSTD – 2016, Google Draco 3D – 2017, DropBox DivANS – 2018.
- FSE (tANS) je typicky kombinováno s LZ (statický blok) protože aktualizace pravděpodobností je drahá operace stejně jako u adaptivního Huffmanova kódování.

Demo: <http://demonstrations.wolfram.com/DataCompressionUsingAsymmetricNumeralSystems/>



Poslední vývoj v oblasti bezztrátové komprese

Komprese genomu (bioinformatika)

- Zdokonalování technologií pro sekvencování genomu vedlo k dramatickému snížení nákladů, masivnímu rozšíření a tím k nárůstu nároků na kapacitu úložiště. Např. 1000 Genome Project vyprodukoval 260 TB dat (2012).
- Uchovávání a přenos takového množství dat představuje závažný problém, který vedl na vytvoření vysoce výkonných bezztrátových kompresních nástrojů specificky navržených pro potřebu komprese genomických dat a rychlé vyhledávání v nich.
- Využívá se toho, že se komprimuje více genomů stejného druhu případně znalosti polymorfismu (DNAZIP/GenomeZIP)

Metoda	Kódování, princip	Kompresní poměr
Genome Differential Compressor (2011)	LZ77/Huffman coding	180 – 250
DNAzip (2009)	Huffman coding	~750
GenomeZip (2013)	Entropy coding	~1200
CRAM 3.0 DNA compressor (2019)	Deflate, rANS	cca 1.6x oproti SAM formátu

Poslední vývoj v oblasti bezeztrátové komprese

HTTP komprese (přenos dat, internet)

- HTTP komprese se používá k zvýšení přenosové rychlosti a zefektivnění využití propustnosti linek (stejnou linkou je možné za stejný čas přenést vyšší objem dat)
- HTTP data jsou komprimovaná serverem předtím, nežli jsou odeslána klientovi. Typicky se používá metoda gzip / deflate (1991) ta však v některých případech nevyhovuje – např. WOFF fonty.
- V roce 2015 Google zavedl bezeztrátovou metodu Brotli, která kombinuje LZ77, Huffmanův kód a model kontextu 2. řádu.
- Metoda má rychlost srovnatelnou s deflate, ale vyšší účinnost v důsledku čehož dojde k redukci doby načtení stránky (dle Google každá sekunda zpoždění v načítání způsobí 7% pokles konverze; stránky načítající se déle než 3 sekundy opustí přes 40% lidí).
- Na enwik8 je Brotli:9 rychlejší o 28% v dekódování než deflate:9 a kompresní poměr dává o 13-21% vyšší.

×	Headers	Preview	Response
▼ General			
Request URL: https://www.google			
Request Method: GET			
Status Code: 200			
Remote Address: [2a00:1450:4014			
Referrer Policy: no-referrer-when			
▼ Response Headers			
alt-svc: quic=":443"; ma=2592000;			
cache-control: private, max-age=0			
content-encoding: <u>br</u>			
content-length: 72755			
content-type: text/html; charset			
date: Tue, 05 Feb 2019 20:35:13			
expires: -1			
server: gws			

Poslední vývoj v oblasti bezeztrátové komprese

ad HTTP komprese (přenos dat, internet)

- Ačkoliv se jedná relativně o novinku, je Brotli podporováno
 - na straně serverů: Apache, Nginx, IIS,
 - prohlížečů: Chrome, Firefox, Edge, Opera, Safari, ...
 - ale také CDN: Cloudflare, KeyCDN, CDN77, MaxCND, ...

Brotli Accept-Encoding/Content-Encoding OTHER

More effective lossless compression algorithm than gzip and deflate.

Current aligned Usage relative Date relative Apply filters Show all ?

IE	Edge	Firefox	Chrome	Safari	Opera	iOS Safari	Opera Mini	Android Browser	Opera Mobile	Chrome for Android	Firefox for Android	UC Browser for Android	Samsung Internet	QQ Browser	Balc Brow
			4-48												
			49		10-35										
	12-14	2-43	50	3.1-10.1	36-37	3.2-10.3							4		
6-10	15-18	44-71	51-78	11-12.1	38-63	11-13.1		2.1-4.4.4	12-12.1				5-9.2		
11	79	72	79	13	64	13.2	all	76	46	79	68	12.12	10.1	1.2	7.1
		73-74	80-82	TP		13.3									

Brotli

Speed up page load times for your visitor's HTTPS traffic by applying Brotli compression.

On

API Help

Cloudflare applies Brotli compression to help speed up page load times for your visitors. Cloudflare will select Brotli compression as the preferred content encoding method if multiple compression methods are supported by the client. If the client does not indicate that Brotli compression is supported, then gzip compression will be applied. [What will Cloudflare compress?](#)

Poslední vývoj v oblasti bezztrátové komprese

Obecná kompresní metoda

- V roce 2016 (Collet, Facebook) byla navržena a uvolněna referenční implementace bezztrátového algoritmu, který vylepšuje populární metodu gzip (1991)
- V porovnání s gzip má metoda podobou účinnost avšak dosahuje mnohem vyšší rychlosti co se týče komprese i dekomprese.
- Metoda je založena na LZ77 s velkým vyhledávacím oknem a entropickém kódování Finite state entropy s využitím buď tANS (2014) nebo Huffmanova kódování
- V roce 2018 standardizováno pro HTTP (RFC8478) Content-coding: zstd, není zatím podporováno prohlížeči
- Zatímco Brotli je vhodné pro statický obsah, který je poskytován mnoha klientům a kde tudíž nevadí delší doba komprese, Zstd je výhodné pro low-end a dynamický obsah, který je typicky určen pro jedno zobrazení.

Poslední vývoj v oblasti ztrátové komprese

Komprese videa

- Aby bylo možné udržet požadavky na vysokou kvalitu obrazu, byl nedávno navržen standard High Efficiency Video Coding (HEVC schválený 2013 a uvolněný 2015) známý jako H.265, který nahrazuje rozšířený standard pro kompresi videa H.264 (MPEG-4).
- HEVC byl navržen s cílem zvýšit efektivitu kódování oproti H.264, tj. redukovat bitrate alespoň na polovinu při zachování obrazové kvality, za cenu zvýšení výpočetních nároků.
- Oproti H264 používajícímu makrobloky pevné velikosti 16x16 pixelů, které jsou následně komprimovány, H265 používá bloky o proměnné velikosti až do 64x64 pixelů. Obraz je rozdělen dle úrovně detailů v dané lokalitě.

H.264



HEVC



Poslední vývoj v oblasti ztrátové komprese

ad Komprese videa

- HEVC umožňuje dosáhnout kompresního poměru až 1:1000.
- Využívá bezztrátové kódování Context-adaptive binary arithmetic coding (CABAC) podobně jako v H264, avšak optimalizované na rychlost při zachování podobného kompresního poměru. Vylepšena byla práce s kontextem.
- Pro kompresi se využívají vylepšené techniky známé z H264: Intra prediction, Motion compensation, Motion vector prediction.
- Reziduum (tj. chyba predikce) je transformována pomocí transformace, která je upravenou variantou diskrétní kosinové a sinové transformace DCT/DST.
- HEVC/H.265 se používá v DVB-T2, které v ČR běží od 2017.

Komprese obrazu

- JPEG 2000 založený na využití WT



Základní kompresní techniky

- Základní bezeztrátové kompresní techniky:
 - Kódování délek/délkou sledů (Run-length coding)
 - Kódování založené na pravděpodobnosti (Statistical coding)
 - Slovníkové metody (Dictionary-based coding)
 - Transformační kódování (Move-to-Front, Burrows-Wheeler)

- Základní ztrátové kompresní techniky:
 - Kvantizace (Quantisation)
 - Transformační kódování (Transform-based coding)
 - Predikce pohybu (Motion estimation)

Základní kompresní techniky

- **Kódování délek/délkou sledů (Run-length coding)**
 - Základní myšlenkou je nahradit řadu po sobě jdoucích symbolů dvojicí, která učtuje počet opakování daného symbolu
 - Příklad: Řetězec ABBBBBBBCC může být reprezentován jako A r7 B r2 C, kde r7 a r2 jsou speciální značky značící sedm a dvě opakování následujícího symbolu.
- **Kódování založené na pravděpodobnosti (Statistical coding)**
 - Využívá statistickou informaci a nahrazuje symboly kódem s proměnnou délkou
- **Slovníkové metody (Dictionary-based coding)**
 - Sekvence symbolů vstupní zprávy je nahrazena odkazy do slovníku. Slovník může být pevný nebo dynamicky vytvářený.

Základní kompresní techniky

■ Kvantizace (Quantisation)

- Základní myšlenkou je aproximovat data, tj. převést je do jednodušší podoby.
- Příklad: Uvažujme sekvenci integerů (7, 223, 15, 28, 64, 37, 145). Pokud bychom potřebovali uchovat přesné hodnoty, potřebujeme 8 bitů na každou hodnotu. Pokud bychom však uchovávali pouze zbytek po dělení šestnácti zaokrouhlený na nejbližší celé číslo (0, 14, 1, 2, 4, 2, 9), stačí nám k uchování pouze 4 bity.

■ Transformační kódování (Transform-based coding)

- Data jsou transformována do jiné domény např. pomocí DCT / DFT / WT, kde je možné provést efektivnější kódování neboť řada koeficientů je blízkých nule.

■ Predikce pohybu (Motion prediction)

- Typicky používáno pro kompresi videa, kdy objekty (např. bloky 8x8 pixelů) jsou v jednotlivých snímcích kódovány jako odkazy na určitý objekt (typicky na jiné pozici) v předchozím snímku.

Doporučená studijní literatura

Knihy

- David Salomon: Data Compression, The Complete Reference, Third Edition, Springer, 2004, 899 stran
- Khalid Sayood: Introduction to Data Compression, Third Edition, Morgan Kaufmann, 2006, 680 stran

Studijní opora

- Vladimír Drábek: Kódování a komprese dat